ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN I

Toán Ứng Dụng và Thống Kê

Ma Trận
và Cơ Sở Tính Toán Khoa Học

Giảng viên hướng dẫn: Th.S Lê Nhựt Nam
Th.S Võ Nam Thục Đoan

Nhóm sinh viên: 24120506 - Phan Thế Minh Trí
24120168 - Đoàn Ngọc Bảo
24120192 - Trần Nguyễn Quốc Khánh
24120457 - Huỳnh Chí Thoại
24120117 - Nguyễn Quang Phát

TP. Hồ Chí Minh, Tháng 4 năm 2026

Mục lục

Bảng phân công công việc **iii**

1 Phần 1: Phép thử Gauss và các Ứng Dụng **1**
1.1 Phương pháp khử Gauss và Gauss-Jordan 1
1.1.1 Cơ sở lý thuyết 1
1.1.2 Phân tích thuật toán Gauss và Gauss Jordan cùng mã nguồn cài đặt chi tiết (gaussian_eliminate(A, b) và back_substitution(A, b)) 3
1.2 Tính Định Thức qua phép khử Gauss 8
1.2.1 Cơ sở lý thuyết 8
1.2.2 Phân tích thuật toán tính định thức bằng phép khử gauss và mã nguồn cài đặt chi tiết (determinant(A) 8
1.3 Tìm Ma Trận Nghịch Đảo bằng phương pháp Gauss-Jordan 10
1.3.1 Cơ sở lý thuyết 10
1.3.2 Phân tích thuật toán tìm nghịch đảo bằng phép khử gauss và mã nguồn cài đặt chi tiết (inverse(A)) 11
1.4 Hạng Ma Trận và Cơ Sở 12
1.4.1 Cơ sở lý thuyết 12
1.4.2 Phân tích thuật toán tìm cơ sở cho các không gian và mã nguồn cài đặt chi tiết (rank_and_basic(A)) 14

2 Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim **17**
2.1 Cơ sở lý thuyết 17
2.1.1 Ý nghĩa hình học (Nền tảng trực quan hóa): 18
2.1.2 Mối liên hệ với bài toán chéo hóa ma trận đối xứng: 18
2.2 Mô tả chi tiết thuật toán SVD 19
2.3 Cài đặt thuật toán với Python 19
2.4 Phân tích độ phức tạp và Tính ổn định số học 20
2.5 Trực Quan Hóa với Manim 21
2.6 Tiêu Chí Đánh Giá — Phần 2 21

3 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng **21**
3.1 Mục tiêu của Phần 3 21
3.2 Chuẩn hóa đầu ra cho các phương pháp giải hệ 22
3.3 Các phương pháp giải hệ được sử dụng 22
3.3.1 Phương pháp Gauss-Jordan tái sử dụng từ Phần 1 23
3.3.2 Phương pháp phân rã SVD tái sử dụng từ Phần 2 23
3.3.3 Phương pháp lặp Gauss-Seidel 23
3.4 Danh sách solver sử dụng cho benchmark 23
3.5 Thiết kế dữ liệu thử nghiệm 24
3.6 Quy trình benchmark 24
3.7 Các tiêu chí đánh giá 25
3.8 Lưu kết quả và phục vụ phân tích tiếp theo 25
3.9 Nhận xét định hướng cho phần thực nghiệm 26

4 Kết luận **26**
4.1 Kết quả đạt được 26
4.1.1 Part 1 26
4.1.2 Part 2 26
4.1.3 Part 3 27
4.2 Một số khó khăn trong quá trình thực hiện đồ án 27
4.2.1 Part 1 27
4.2.2 Part 2 28
4.2.3 Part 3 28
4.3 Bài học rút ra 28
4.3.1 Part 1 28
4.3.2 Part 2 28
4.3.3 Part 3 29

Tài liệu tham khảo **29**

Bảng phân công công việc

STT	Công việc	Thành viên
1	Cài đặt các hàm, viết demo, viết báo cáo (Part 1)	Phan Thế Minh Trí
2	Cài đặt thuật toán Phân rã SVD (Part 2)	Huỳnh Chí Thoại
3	Cài đặt Giá Trị riêng, Vector riêng và chéo hóa ma trận	Trần Nguyễn Quốc Khánh
4	Xây dựng sườn và quản lý mã nguồn GitHub, phát triển video Manim trực quan, phân công nhiệm vụ, kiểm tra dữ liệu và hỗ trợ chỉnh sửa các phần chưa đúng của nhóm.	Đoàn Ngọc Bảo
5	Tổng hợp các phương pháp giải hệ phương trình $Ax = b$ và chạy thực nghiệm với ma trận ngẫu nhiên (Part 3)	Trần Nguyễn Quốc Khánh
6	Trình bày phân tích và trực quan hóa dữ liệu (Part 3)	Huỳnh Chí Thoại
7	Kiểm lỗi, tinh chỉnh, hỗ trợ cài đặt mã nguồn (Part 1, Part 3), tổng hợp và viết hướng dẫn clone repo và chạy lại project (README.md)	Nguyễn Quang Phát

Giới thiệu Đồ Án

Bài làm đồ án này được thực hiện bởi nhóm 1, Toán Ứng Dụng và Thống Kê 24CTT3. Tập trung vào ba nhóm kỹ thuật cốt lõi của đại số tuyến tính số trên máy tính:

1. Phép khử Gauss — nền tảng của việc giải hệ phương trình tuyến tính, tính định thức, nghịch đảo và hạng ma trận.
2. Phân rã ma trận (Matrix Decompositions) và Chéo hóa (Diagonalization) — hiểu bản chất hình học và ứng dụng trực quan hóa.
3. Ổn định số học và chi phí tính toán — đánh giá thực nghiệm các phương pháp số trong thực tế.

Qua đồ án này, chúng em đã hoàn tất mọi yêu cầu của đồ án. Chúng em không chỉ nắm vững nền tảng toán học của những vấn đề trên mà còn có khả năng tự cài đặt lại từ đầu bằng Python, trực quan hóa bằng Manim, và phân tích thực nghiệm về hiệu năng thuật toán.

1 Phần 1: Phép thử Gauss và các Ứng Dụng

Trong phần này, chúng em sẽ tập trung vào Phép khử Gauss và các ứng dụng cơ bản của phép khử Gauss trong Đại Số Tuyến Tính nói chung. Các vấn đề được đề cập đến bao gồm:

- Phương pháp khử Gauss và Gauss-Jordan
- Tính Định Thức Ma Trận thông qua phương pháp khử Gauss
- Tìm Ma Trận Nghịch Đảo bằng phương pháp Gauss-Jordan
- Tìm Hạng của Ma Trận và các Cơ Sở Hàng, Cột.

Ở mỗi chủ đề, chúng em sẽ trình bày cơ sở lý thuyết, mã giả và đi vào phân tích mã nguồn chi tiết một cách song song với nhau.

1.1 Phương pháp khử Gauss và Gauss-Jordan

1.1.1 Cơ sở lý thuyết

Xét hệ phương trình tuyến tính $\mathbf{Ax} = \mathbf{b}$ với $\mathbf{A} \in \mathbb{R}^{m\times n}, \mathbf{x} \in \mathbb{R}^n, \mathbf{b} \in \mathbb{R}^m$.
Định nghĩa 1.1 (Ma trận tăng cường). *Ma trận tăng cường (augmented matrix) của hệ* $\mathbf{Ax} = \mathbf{b}$ *là:*

$$[\mathbf{A} \mid \mathbf{b}] = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{pmatrix}$$

Định nghĩa 1.2 Cho $A \in M_{m\times n}(\mathbb{R})$. Ta gọi phép biến đổi sơ cấp trên dòng, viết tắt là phép BĐSCTD trên A, là một trong ba loại biến đổi sau:

- **Loại 1.** Hoán vị hai dòng i và j $(i \neq j)$.

  Ký hiệu: $d_i \leftrightarrow d_j$

- **Loại 2.** Nhân dòng i với một số $\alpha \neq 0$.

  Ký hiệu: $\alpha d_i \to d_i$ (hoặc αd_i)

- **Loại 3.** Cộng vào dòng i với β lần dòng j $(j \neq i)$.

  Ký hiệu: $d_i + \beta d_j \to d_i$ (hoặc $d_i + \beta d_j$)

Nhận xét 1. Các phép biến đổi sơ cấp dòng thực chất là các phép biến đổi đại số cơ bản trên hệ phương trình, vì thế nó không làm thay đổi tập nghiệm của hệ phương trình $\mathbf{Ax} = \mathbf{b}$

Định nghĩa 1.3 Cho $A \in M_{m\times n}(\mathbb{R})$. Phần tử khác 0 đầu tiên của một dòng kể từ bên trái qua được gọi là **phần tử cơ sở** (pivot) của dòng đó. Một ma trận được gọi là **ma trận bậc thang** nếu nó thỏa mãn 2 tính chất sau:

1. Các dòng bằng không (nếu có) luôn nằm dưới cùng;
2. Phần tử cơ sở của dòng dưới luôn nằm bên phải so với phần tử cơ sở của dòng trên.

Như vậy ma trận bậc thang có dạng:

$$\begin{pmatrix}
0 & \dots & 0 & \mathbf{a_{1k_1}} & \dots & \dots & a_{1k_2} & \dots & \dots & a_{1k_r} & \dots & a_{1n} \\
0 & \dots & 0 & 0 & \dots & 0 & \mathbf{a_{2k_2}} & \dots & \dots & a_{2k_r} & \dots & a_{2n} \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
0 & \dots & 0 & 0 & \dots & 0 & 0 & \dots & 0 & \mathbf{a_{rk_r}} & \dots & a_{rn} \\
0 & \dots & 0 & 0 & \dots & 0 & 0 & \dots & 0 & 0 & \dots & 0 \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
0 & \dots & 0 & 0 & \dots & 0 & 0 & \dots & 0 & 0 & \dots & 0
\end{pmatrix}$$

Sau một số phép biến đổi sơ cấp dòng trên ma trận A, ta thu được ma trận bậc thang A' thì ta nói A' là dạng ma trận bậc thang (REF) của A.

Định nghĩa 1.4 Ma trận A được gọi là *ma trận bậc thang rút gọn* nếu thỏa mãn 3 điều kiện sau:

1. A là ma trận bậc thang.
2. Các phần tử cơ sở (pivot) đều bằng 1.
3. Trên cột có chứa phần tử cơ sở, các phần tử không phải phần tử cơ sở đều bằng 0.

Sau một số phép biến đổi sơ cấp dòng trên ma trận A, ta thu được ma trận bậc thang rút gọn A' thì ta nói A' là dạng ma trận bậc thang rút gọn (RREF) của A.

Nhận xét 2. Từ nhận xét 1, khi sử dụng một loạt các phép biến đổi sơ cấp dòng biến đổi ma trận tăng cường $[\mathbf{A} \mid \mathbf{b}]$ về dạng ma trận bậc thang rút gọn $[\mathbf{A}' \mid \mathbf{b}']$. Ta thực hiện bỏ đi tất cả các cột toàn 0 của $[\mathbf{A}' \mid \mathbf{b}']$ thì:

1. Nếu vẫn tồn tại một hàng i nào đó của A' toàn 0 thì phương trình $Ax = b$ vô nghiệm.

2. Nếu lúc này A' là một ma trận đơn vị thì hệ phương trình $Ax = b$ có nghiệm duy nhất là b'.

3. Nếu lúc này A' có số cột nhiều hơn số hàng thì hệ phương trình $Ax = b$ có vô số nghiệm. Khi đó, gọi ẩn tự do là các ẩn ứng với các cột không chứa pivot, ta có thể biểu diễn tập nghiệm thông qua các ẩn tự do này bằng cách thế ngược từ dưới lên.

1.1.2 Phân tích thuật toán Gauss và Gauss Jordan cùng mã nguồn cài đặt chi tiết (gaussian_eliminate(A, b) và back_substitution(A, b))

Từ các cơ sở lý thuyết trên, ta có thuật toán đưa một ma trận A về dạng bậc thang như sau:

THUẬT TOÁN KHỬ GAUSS

Bước 1. Khởi tạo chỉ số: $i := 1, j := 1$.

Bước 2. Điều kiện dừng: Nếu $i > m$ hoặc $j > n$ thì **Kết thúc**.

Bước 3. Kiểm tra phần tử a_{ij}:

- Nếu $a_{ij} \neq 0$: Thực hiện phép biến đổi dòng khử các phần tử dưới nó:

$$d_k \leftarrow d_k - \frac{a_{kj}}{a_{ij}} d_i \quad (\forall k > i)$$

  Sau đó tăng $i := i + 1, j := j + 1$ và quay lại **Bước 2**.
- Nếu $a_{ij} = 0$: Chuyển sang **Bước 4**.

Bước 4. Xử lý khi gặp số 0:

- Nếu tồn tại $k > i$ sao cho $a_{kj} \neq 0$: Đổi chỗ hai dòng $d_i \leftrightarrow d_k$, sau đó quay lại **Bước 3**.
- Nếu $a_{kj} = 0$ với mọi $k > i$: Tăng $j := j + 1$ (bỏ qua cột này) và quay lại **Bước 2**.

Tương tự thuật toán trên, ta có thuật toán Gauss-Jordan để đưa ma trận về dạng bậc thang rút gọn như sau:

THUẬT TOÁN GAUSS-JORDAN

Bước 1. Khởi tạo: $i := 1, j := 1$.

Bước 2. Dừng nếu $i > m$ hoặc $j > n$.

Bước 3. Nếu $a_{ij} \neq 0$:

- **Chuẩn hóa dòng trụ:** Biến phần tử cơ sở thành số 1:

$$d_i \leftarrow \frac{1}{a_{ij}} d_i$$

- **Khử toàn bộ cột:** Biến mọi phần tử khác trên cột j thành 0:

$$d_k \leftarrow d_k - a_{kj} d_i \quad (\forall k \neq i)$$

- Tăng $i := i + 1, j := j + 1$ và quay lại **Bước 2**.

Bước 4. Nếu $a_{ij} = 0$:

- Nếu có dòng $k > i$ mà $a_{kj} \neq 0$: Đổi chỗ $d_i \leftrightarrow d_k$, quay lại **Bước 3**.
- Nếu không (cả cột dưới đều bằng 0): Tăng $j := j + 1$, quay lại **Bước 2**.

Dựa trên thuật toán Gauss-Jordan đã trình bày, ta có cài đặt hàm thực hiện nhận vào 2 ma trận A, b, sau đó tạo ma trận tăng cường và đưa về dạng RREF dưới đây. Hàm sẽ trả về A và b sau khi đã thực hiện thuật toán.

```python
def gaussian_eliminate(A, b):
    # Ham nhan 2 ma tran A, b, tao ma tran tang cuong va dua ve dang RREF
    n, m = A.rows, A.cols
    aug_data = [A.data[i] + b.data[i] for i in range(n)]
    # Tao ma tran tang cuong
    Ab = Matrix(aug_data, "Augmented")

    pivot_row = 0
    for j in range(m):
        if pivot_row >= n: break

        # Tim hang co phan tu lon nhat de tranh sai so
        max_idx = pivot_row
        for k in range(pivot_row + 1, n):
            if abs(Ab.data[k][j]) > abs(Ab.data[max_idx][j]):
                max_idx = k

        # Kiem tra neu cot toan so 0 (hoac gan bang 0)
        if abs(Ab.data[max_idx][j]) < 1e-8: continue

        Ab.swap_rows(pivot_row, max_idx)

        # Chuan hoa dong tru ve 1
```

```
        pivot_val = Ab.data[pivot_row][j]
        Ab.multiply_row_with_real_number(pivot_row, 1.0 /
    pivot_val)

        # Khu cac phan tu khac tren cot j ve 0
        for i in range(n):
            if i != pivot_row:
                factor = -Ab.data[i][j]
                Ab.add_multiple_of_row(i, pivot_row, factor)

        pivot_row += 1

    # Tach ma tran tang cuong va cap nhat lai A, b
    A.data = [row[:m] for row in Ab.data]
    b.data = [row[m:] for row in Ab.data]
    return A, b
```

Listing 1: Hàm gaussian_eliminate

Từ nhận xét 2, ta lập được thuật toán giải hệ phương trình như sau:

THUẬT TOÁN THẾ NGƯỢC (BACK SUBSTITUTION)

Bước 1. Kiểm tra tính tương thích: Duyệt qua từng hàng của ma trận tăng cường $[A \mid b]$ từ hàng cuối lên trên:

- Nếu tồn tại hàng có dạng $[0 \dots 0 \mid k]$ với $k \neq 0$: Kết luận hệ phương trình vô nghiệm và **Kết thúc**.

Bước 2. Phân loại ẩn số: Dựa trên các phần tử chốt (pivots) của ma trận bậc thang:

- **Ẩn cơ bản:** Các ẩn ứng với cột chứa phần tử chốt.
- **Ẩn tự do:** Các ẩn còn lại. Gán các ẩn này bằng tham số: $x_j := t_1, t_2, \dots$

Bước 3. Thực hiện tính toán từ dưới lên: Duyệt từng hàng i từ hàng cuối cùng có chứa phần tử chốt ngược lên hàng 1:

- Xác định ẩn chốt x_{pivot} tại cột j của hàng i.
- Cô lập ẩn chốt bằng công thức thế ngược:

$$x_j = \frac{1}{a_{ij}} \left(b_i - \sum_{k=j+1}^{m} a_{ik} x_k \right)$$

- **Xử lý biểu thức:**
  - Nếu các x_k là số thực: Thực hiện phép tính trừ trực tiếp vào hằng số.
  - Nếu các x_k chứa tham số (t): Kết hợp chúng vào biểu thức đại số của x_j.

Bước 4. Tổng hợp nghiệm: Gộp kết quả của tất cả các ẩn (cả ẩn cơ bản và ẩn tự do) thành vector nghiệm tổng quát.

Bước 5. Kết thúc: Trả về tập nghiệm dưới dạng số thực hoặc biểu thức chứa tham số.

```python
def back_substitution(A, b):
    n, m = A.rows, A.cols
    if b.cols != 1:
        return "Loi: b phai la ma tran cot."

    # 1. Kiem tra vo nghiem
    for i in range(n):
        if all(abs(A.data[i][j]) < 1e-8 for j in range(m)) and abs(b.data[i][0]) > 1e-8:
            return "He phuong trinh vo nghiem."

    # 2. Xac dinh Pivots va An tu do
    pivot_col = {}
    is_pivot_column = [False] * m
    for i in range(n):
```

```python
        for j in range(m):
            if abs(A.data[i][j]) > 1e-8:
                pivot_col[i] = j
                is_pivot_column[j] = True
                break

    res = [None] * m
    t_idx = 1
    for j in range(m):
        if not is_pivot_column[j]:
            res[j] = f"t{t_idx}"
            t_idx += 1

    # 3. Qua trinh The nguoc
    for i in range(n - 1, -1, -1):
        if i not in pivot_col: continue

        curr_col = pivot_col[i]
        pivot_val = A.data[i][curr_col]
        constant_part = b.data[i][0]
        expression_parts = []

        for j in range(curr_col + 1, m):
            coeff = A.data[i][j]
            if abs(coeff) > 1e-8:
                if isinstance(res[j], (int, float)):
                    constant_part -= coeff * res[j]
                else:
                    reduced_coeff = -coeff / pivot_val
                    sign = "+" if reduced_coeff > 0 else "-"
                    val = abs(round(reduced_coeff, 4))
                    expression_parts.append(f"{sign} {val}*{res[j]}")

        # 4. Tong hop nghiem
        final_constant = round(constant_part / pivot_val, 4)
        if not expression_parts:
            res[curr_col] = final_constant
        else:
            expr = " ".join(expression_parts)
            if final_constant == 0:
                res[curr_col] = expr[2:] if expr.startswith("+ ") else expr
            else:
                res[curr_col] = f"{final_constant} {expr}"

    return res
```

Listing 2: Hàm back_substitution

1.2 Tính Định Thức qua phép khử Gauss

1.2.1 Cơ sở lý thuyết

Nhận xét 3. Cho $A, A' \in M_n(\mathbb{R})$. Khi đó:

(i) Nếu $A \xrightarrow[i \neq j]{d_i \leftrightarrow d_j} A'$ thì $|A'| = -|A|$;

(ii) Nếu $A \xrightarrow{\alpha d_i} A'$ thì $|A'| = \alpha |A|$ hay $|A| = \frac{1}{\alpha}|A'|$;

(iii) Nếu $A \xrightarrow[i \neq j]{d_i + \beta d_j} A'$ thì $|A'| = |A|$.

Nhận xét 4. Cho $A \in M_n(\mathbb{R})$. Nếu A là ma trận tam giác thì $|A|$ được tính bằng tích các phần tử trên đường chéo, nghĩa là:

$$|A| = a_{11} \times a_{22} \times \cdots \times a_{nn}$$

1.2.2 Phân tích thuật toán tính định thức bằng phép khử gauss và mã nguồn cài đặt chi tiết (determinant(A)

Dựa vào cơ sở lý thuyết trên, ta sẽ tìm cách đưa ma trận A về dạng ma trận bậc thang và tính định thức của ma trận này, sau đó suy ngược lại định thức của A.

THUẬT TOÁN TÍNH ĐỊNH THỨC

Bước 1. Khởi tạo: Sao chép ma trận A sang một bản tạm để tránh thay đổi dữ liệu gốc. Khởi tạo biến tích lũy dấu $det_sign := 1$.

Bước 2. Lặp qua từng cột i từ 1 đến n:

- **Tìm phần tử chốt (Partial Pivoting):** Xác định hàng $k \geq i$ có giá trị tuyệt đối $|a_{ki}|$ lớn nhất để đảm bảo tính ổn định số học.
- **Xử lý đổi chỗ hàng:** Nếu $k \neq i$:
  - Đổi chỗ hai dòng $d_i \leftrightarrow d_k$.
  - Đảo dấu biến tích lũy: $det_sign \leftarrow -det_sign$.
- **Kiểm tra suy biến:** Nếu phần tử tại đường chéo $a_{ii} \approx 0$: Kết luận định thức bằng 0 và **Kết thúc**.

Bước 3. Thực hiện phép khử: Triệt tiêu các phần tử bên dưới đường chéo chính tại cột i:

$$d_k \leftarrow d_k - \frac{a_{ki}}{a_{ii}} d_i \quad (\forall k > i)$$

Sau đó quay lại **Bước 2** cho cột tiếp theo.

Bước 4. Tính toán giá trị cuối cùng: Sau khi đưa ma trận về dạng tam giác trên, định thức được tính bằng tích của biến dấu và các phần tử trên đường chéo chính:

$$\det(A) = det_sign \times \prod_{i=1}^{n} a_{ii}$$

Bước 5. Kết thúc: Trả về giá trị định thức (có thể làm tròn để khử sai số dấu phẩy động).

Dựa trên thuật toán này, ta có mã nguồn chi tiết như dưới đây. Hàm nhận vào ma trận A và trả về định thức của A.

```python
def determinant(A):
    if A.rows != A.cols:
        raise ValueError("Chi tinh duoc dinh thuc cua ma tran vuong.")

    n = A.rows
    A_copy = A.copy()
    det = 1.0

    for i in range(n):
        # 1. Tim phan tu chot lon nhat (Partial Pivoting)
        max_row = i
        for k in range(i + 1, n):
            if abs(A_copy.data[k][i]) > abs(A_copy.data[max_row][i]):
                max_row = k

```

```
        # 2. Doi cho hang va doi dau dinh thuc
        if i != max_row:
            A_copy.swap_rows(i, max_row)
            det *= -1

        # 3. Neu duong cheo bang 0, dinh thuc bang 0
        if abs(A_copy.data[i][i]) < 1e-10:
            return 0.0

        # 4. Khu cac phan tu phia duoi duong cheo chinh
        for k in range(i + 1, n):
            factor = -A_copy.data[k][i] / A_copy.data[i][i]
            A_copy.add_multiple_of_row(k, i, factor)

    # 5. Tich cac phan tu tren duong cheo chinh
    for i in range(n):
        det *= A_copy.data[i][i]

    return round(det, 4)
```

Listing 3: Hàm determinant sử dụng khử Gauss

1.3 Tìm Ma Trận Nghịch Đảo bằng phương pháp Gauss-Jordan

1.3.1 Cơ sở lý thuyết

Định nghĩa 1.5 Cho $A \in M_n(\mathbb{R})$. Ta nói A khả nghịch nếu tồn tại ma trận B sao cho

$$AB = BA = I_n$$

Nếu B thỏa điều kiện trên thì B được gọi là **ma trận nghịch đảo** của A (kí hiệu là A^{-1}). Ma trận nghịch đảo của một ma trận khả nghịch là duy nhất. Ta ký hiệu ma trận nghịch đảo của A là A^{-1}

Nhận xét 5. Cho $A \in M_n(\mathbb{R})$. Khi đó các khẳng định sau tương đương:

(i) A khả nghịch.

(ii) $A \sim I_n$ (ma trận A tương đương dòng với ma trận đơn vị).

(iv) Tồn tại các phép biến đổi sơ cấp trên dòng (BĐSCTD) $\phi_1, \ldots, \phi_k$ biến ma trận A thành ma trận đơn vị I_n:

$$A \xrightarrow{\phi_1} A_1 \xrightarrow{\phi_2} \ldots \xrightarrow{\phi_k} A_k = I_n$$

Hơn nữa, khi đó qua chính các phép BĐSCTD $\phi_1, \ldots, \phi_k$ đó, ma trận đơn vị I_n sẽ biến thành ma trận nghịch đảo A^{-1}:

$$I_n \xrightarrow{\phi_1} B_1 \xrightarrow{\phi_2} \ldots \xrightarrow{\phi_k} B_k = A^{-1}$$

1.3.2 Phân tích thuật toán tìm nghịch đảo bằng phép khử gauss và mã nguồn cài đặt chi tiết (inverse(A))

Từ Nhận xét 5, ta có thuật toán sau: Lập ma trận ghép $(A|I_n)$ và dùng các phép biến đổi sơ cấp trên dòng (BĐSCTD) để đưa A về dạng ma trận bậc thang rút gọn:

$$(A|I_n) \xrightarrow{\phi_1} (A_1|B_1) \xrightarrow{\cdots} \xrightarrow{\phi_p} (A_p|B_p) \xrightarrow{\cdots}$$

- **Trường hợp 1:** Trong quá trình biến đổi, nếu xuất hiện ma trận A_p có ít nhất một dòng hay một cột bằng không thì A không khả nghịch.
- **Trường hợp 2:** Ngược lại, ma trận cuối cùng của dãy trên có dạng $(I_n|B)$. Khi đó A khả nghịch và $A^{-1} = B$.

THUẬT TOÁN TÌM MA TRẬN NGHỊCH ĐẢO (GAUSS-JORDAN)

Bước 1. Khởi tạo: Ghép ma trận đơn vị I vào bên phải ma trận A để được ma trận tăng cường $[A \mid I]$. Đặt chỉ số hàng $i := 1$.

Bước 2. Điều kiện dừng: Nếu $i > n$ (với n là cấp của ma trận) thì sang **Bước 5**.

Bước 3. Kiểm tra phần tử chốt a_{ii}:

- Nếu $a_{ii} \neq 0$: Thực hiện các phép biến đổi:
  - Chuẩn hóa dòng i: $d_i \leftarrow \frac{1}{a_{ii}} d_i$.
  - Khử toàn bộ cột i: $d_k \leftarrow d_k - a_{ki} d_i \quad (\forall k \neq i)$.

  Sau đó tăng $i := i + 1$ và quay lại **Bước 2**.
- Nếu $a_{ii} = 0$: Chuyển sang **Bước 4**.

Bước 4. Xử lý khi gặp số 0:

- Nếu tồn tại $k > i$ sao cho $a_{ki} \neq 0$: Thực hiện đổi chỗ hai dòng $d_i \leftrightarrow d_k$ và quay lại **Bước 3**.
- Nếu $a_{ki} = 0$ với mọi $k > i$: Kết luận ma trận A không khả nghịch và **Kết thúc**.

Bước 5. Kết thúc: Ma trận bên phải của ma trận tăng cường lúc này chính là A^{-1}.

Từ thuật toán này, ta có cài đặt mã nguồn chi tiết như sau:

```python

def inverse(A):
    # Ham tim nghich dao ma tran bang phuong phap Gauss-Jordan
    if A.rows != A.cols:
        raise ValueError(f"Ma tran {A.name} khong phai ma tran vuong.")

    n = A.rows
    A_copy = A.copy(name=f"{A.name}_tmp")

    # Tao ma tran don vi I
```

```python
    identity_data = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
    I = Matrix(identity_data, "Identity")

    # Thuc hien khu Gauss-Jordan tren ma tran tang cuong [A|I]
    gaussian_eliminate(A_copy, I)

    # Kiem tra neu ve trai khong phai ma tran don vi (suy bien)
    for i in range(n):
        if abs(A_copy.data[i][i] - 1.0) > 1e-10:
            print(f"Ma tran {A.name} la ma tran suy bien (khong co nghich dao).")
            return None

    # 6. Tra ve ket qua la ma tran ben phai
    I.name = f"({A.name})^-1"
    return I
```

Listing 4: Hàm inverse sử dụng phương pháp Gauss-Jordan

1.4 Hạng Ma Trận và Cơ Sở

1.4.1 Cơ sở lý thuyết

Định nghĩa 1.6 Một ma trận A có thể có nhiều dạng bậc thang khác nhau, tuy nhiên tất cả các dạng bậc thang của A đều có cùng số dòng khác không. Ta gọi số dòng khác không này là **hạng của ma trận** A, ký hiệu là $r(A)$.
Nhận xét 6. Cho $A, B \in M_{m\times n}(\mathbb{R})$. Khi đó:

(i) $0 \leq r(A) \leq \min\{m, n\}$.

(ii) $r(A) = 0 \iff A = 0$ (với 0 là ma trận không).

(iii) Nếu $A \sim B$ thì $r(A) = r(B)$.

(iv) $r(A^\top) = r(A)$.

Định nghĩa 1.7 Cho V là một tập hợp với phép toán $+$ và phép nhân vô hướng $\cdot$ của $\mathbb{R}$ với V. Khi đó V được gọi là **không gian vectơ** trên $\mathbb{R}$ nếu mọi $u, v, w \in V$ và mọi $\alpha, \beta \in \mathbb{R}$ thỏa mãn 8 tính chất sau:

1. $u + v = v + u$
2. $(u + v) + w = u + (v + w)$
3. Tồn tại $0 \in V$ sao cho: $u + 0 = 0 + u = u$
4. Tồn tại $u' \in V$ sao cho: $u' + u = u + u' = 0$
5. $(\alpha\beta) \cdot u = \alpha \cdot (\beta \cdot u)$
6. $(\alpha + \beta) \cdot u = \alpha \cdot u + \beta \cdot u$
7. $\alpha \cdot (u + v) = \alpha \cdot u + \alpha \cdot v$

8. $1 \cdot u = u$

Định nghĩa 1.8 Cho $u_1, u_2, \ldots, u_m \in V$. Một **tổ hợp tuyến tính** của $u_1, u_2, \ldots, u_m$ là một vector có dạng:

$$u = \alpha_1 u_1 + \alpha_2 u_2 + \cdots + \alpha_m u_m \quad \text{với } \alpha_i \in \mathbb{R}$$

Khi đó, đẳng thức trên được gọi là **dạng biểu diễn** của u theo các vector $u_1, u_2, \ldots, u_m$.
Định nghĩa 1.9 Cho $u_1, u_2, \ldots, u_m \in V$. Xét phương trình:

$$\alpha_1 u_1 + \alpha_2 u_2 + \cdots + \alpha_m u_m = \mathbf{0} \quad (\star)$$

- Nếu $(\star)$ chỉ có **nghiệm tầm thường** $\alpha_1 = \alpha_2 = \cdots = \alpha_m = 0$ thì ta nói $u_1, u_2, \ldots, u_m$ (hay $\{u_1, u_2, \ldots, u_m\}$) **độc lập tuyến tính**.
- Nếu $(\star)$ có **nghiệm không tầm thường** thì ta nói $u_1, u_2, \ldots, u_m$ (hay $\{u_1, u_2, \ldots, u_m\}$) **phụ thuộc tuyến tính**.

Nói cách khác:

▷ Nếu phương trình $(\star)$ có nghiệm duy nhất thì $u_1, u_2, \ldots, u_m$ độc lập tuyến tính.

▷ Nếu phương trình $(\star)$ có vô số nghiệm thì $u_1, u_2, \ldots, u_m$ phụ thuộc tuyến tính.

Định nghĩa 1.10 Cho V là không gian vector và S là tập con của V. Tập S được gọi là **tập sinh** của V nếu mọi vector của V đều là tổ hợp tuyến tính của S. Khi đó, ta nói S sinh ra V hoặc V được sinh bởi S, ký hiệu $V = \langle S \rangle$.
Định nghĩa 1.11 Cho V là không gian vector và $\mathcal{B}$ là tập con của V. Tập $\mathcal{B}$ được gọi là một **cơ sở** của V nếu thỏa mãn đồng thời hai điều kiện sau:

(i) $\mathcal{B}$ là một tập sinh của V;

(ii) $\mathcal{B}$ độc lập tuyến tính.

Nhận xét 7. Giả sử V có một cơ sở B gồm n vectơ. Khi đó mọi cơ sở khác của V cũng có đúng n vector.
Nhận xét 8. Cho V là một không gian vectơ có $\dim V = n$. Khi đó:

(i) Mọi tập con độc lập tuyến tính có n vectơ đều là cơ sở.

(ii) Mọi tập sinh có n vectơ đều là cơ sở.

Hệ quả. Phương pháp nhận diện cơ sở của không gian V có $dim\ V = n$: Vì $\dim V = n$ nên mọi cơ sở của V phải gồm đúng n vector. Hơn nữa, nếu S là tập con của V và số phần tử của S bằng n ($|S| = n$) thì các khẳng định sau là tương đương:

- S là cơ sở của V;
- S độc lập tuyến tính;
- S là tập sinh của V.

Hay viết dưới dạng toán học:

$$S \text{ là cơ sở} \iff S \text{ độc lập tuyến tính} \iff S \text{ là tập sinh}$$

Định nghĩa 1.12 Cho ma trận $A = (a_{ij}) \in M_{m\times n}(\mathbb{R})$:

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$

Đặt:

$$\begin{aligned} u_1 &= (a_{11}, a_{12}, \dots, a_{1n}) \\ u_2 &= (a_{21}, a_{22}, \dots, a_{2n}) \\ &\vdots \\ u_m &= (a_{m1}, a_{m2}, \dots, a_{mn}) \end{aligned}$$

và $W_A = \langle u_1, u_2, \dots, u_m \rangle$.

Ta gọi $u_1, u_2, \dots, u_m$ là các **vectơ dòng** của A, và W_A được gọi là **không gian dòng** của A.

Nhận xét 9. Nếu A và B là hai ma trận tương đương dòng ($A \sim B$) thì $W_A = W_B$. Nghĩa là hai ma trận tương đương dòng có cùng không gian dòng. Từ đây, ta suy ra rằng giả sử $A \in M_{m\times n}(\mathbb{R})$. Khi đó:

(i) $\dim W_A = r(A)$ (Số chiều của không gian dòng bằng hạng của ma trận A).

(ii) Tập hợp các vectơ khác không trong một dạng bậc thang của A là một cơ sở của không gian dòng W_A.

Nhận xét 10. Các phép biến đổi dòng bảo toàn sự phụ thuộc/độc lập tuyến tính trên không gian cột. Vì thế, trong ma trận bậc thang, các cột chứa phần tử trụ (pivot) luôn độc lập tuyến tính và là "tối đại", nên các cột tương ứng ở ma trận gốc cũng sẽ độc lập tuyến tính tối đại và sinh ra toàn bộ không gian cột.

1.4.2 Phân tích thuật toán tìm cơ sở cho các không gian và mã nguồn cài đặt chi tiết (rank_and_basic(A))

Từ nhận xét 9 ở trên, ta có thuật toán tìm số chiều và cơ sở của không gian dòng như sau:

THUẬT TOÁN TÌM CƠ SỞ CỦA KHÔNG GIAN DÒNG $Row(A)$

Bước 1. Biến đổi về dạng bậc thang: Sử dụng các phép biến đổi sơ cấp trên dòng để đưa ma trận A về dạng bậc thang R_A.

Bước 2. Xác định số chiều ($\dim Row(A)$):

- Số chiều của không gian dòng $Row(A)$ bằng đúng số hàng khác không của ma trận bậc thang R_A.
- Ký hiệu: $\dim(Row(A)) = r(A)$.

Bước 3. Xác định cơ sở của $Row(A)$:

- Tập hợp tất cả các vectơ dòng khác không trong ma trận bậc thang R_A lập thành một cơ sở của không gian dòng $Row(A)$.

Bước 4. Kết thúc: Trả về số chiều và bộ vectơ cơ sở vừa tìm được.

Từ nhận xét 10, ta có thuật toán tìm số chiều và cơ sở không gian cột như sau:

THUẬT TOÁN TÌM CƠ SỞ CỦA KHÔNG GIAN CỘT $Col(A)$

Bước 1. Biến đổi về dạng bậc thang: Sử dụng các phép biến đổi sơ cấp trên dòng để đưa ma trận A về dạng bậc thang R_A.

Bước 2. Xác định số chiều ($\dim Col(A)$):

- Số chiều của không gian dòng C bằng đúng số hàng khác không của ma trận bậc thang R_A.
- Ký hiệu: $\dim(Col(A)) = r(A)$.

Bước 3. Xác định cơ sở của $Col(A)$:

- Tập hợp tất cả các cột của ma trận gốc A chứa pivot lập thành một cơ sở của không gian cột $Col(A)$.

Bước 4. Kết thúc: Trả về số chiều và bộ vectơ cơ sở vừa tìm được.

THUẬT TOÁN TÌM CƠ SỞ CỦA KHÔNG GIAN NGHIỆM $Null(A)$

Bước 1. Giải hệ phương trình tuyến tính thuần nhất $Ax = \mathbf{0}$ để tìm nghiệm tổng quát.

Bước 2. Lần lượt cho bộ ẩn tự do các giá trị $(1, 0, \ldots, 0), (0, 1, \ldots, 0), \ldots, (0, 0, \ldots, 1)$ ta được các nghiệm cơ bản $u_1, u_2, \ldots, u_m$.

Bước 3. Khi đó không gian nghiệm có cơ sở là $\{u_1, u_2, \ldots, u_m\}$.

Bước 4. Kết thúc: Trả về số chiều và bộ vectơ cơ sở vừa tìm được.

Tổng hợp lại, ta thu được thuật toán dưới đây

THUẬT TOÁN XÁC ĐỊNH HẠNG VÀ CƠ SỞ CÁC KHÔNG GIAN

Bước 1. Tiền xử lý: Sao chép ma trận A và đưa về dạng bậc thang rút gọn ($RREF$). Lưu lại danh sách các chỉ số cột chứa phần tử chốt ($pivot_cols$).

Bước 2. Xác định Hạng (Rank):

- Giá trị hạng $r(A)$ chính bằng số lượng phần tử trong danh sách cột chốt: $r = |pivot_cols|$.

Bước 3. Trích xuất cơ sở không gian dòng ($Row(A)$):

- Cơ sở là tập hợp các hàng khác không của ma trận $RREF$ thu được ở Bước 1.

Bước 4. Trích xuất cơ sở không gian cột ($Col(A)$):

- Chọn các cột của **ma trận gốc** A tại các vị trí chỉ số thuộc $pivot_cols$.
- **Lưu ý:** Không sử dụng cột của ma trận $RREF$ vì chúng không bảo toàn không gian cột gốc.

Bước 5. Xây dựng cơ sở không gian nghiệm ($Nul(A)$):

- Xác định các ẩn tự do ứng với các cột không chứa phần tử chốt.
- Với mỗi ẩn tự do x_{free}, tạo một nghiệm đặc biệt bằng cách:
  - Gán $x_{free} := 1$ và các ẩn tự do khác bằng 0.
  - Tính giá trị các ẩn chốt x_{pivot} dựa trên các hệ số tương ứng trong ma trận $RREF$ để thỏa mãn phương trình $Ax = 0$.

Bước 6. Kết thúc: Trả về một cấu trúc dữ liệu gồm Hạng và ba bộ cơ sở đã tìm được.

Mã nguồn cài đặt chi tiết:

```python

def rank_and_basic(A):
    # Ham tinh hang va tim co so cua Row, Column, Null space
    n, m = A.rows, A.cols
    A_copy = A.copy()
    b_dummy = Matrix([[0.0] for _ in range(n)], "zero")

    # 1. Dua ve RREF de phan tich
    gaussian_eliminate(A_copy, b_dummy)

    pivot_cols = []
    row_basis = []

    # 2. Xac dinh cac phan tu chot va Row Basis
    for i in range(n):
        found_pivot = False
```

```
        for j in range(m):
            if abs(A_copy.data[i][j]) > 1e-10:
                if not found_pivot:
                    pivot_cols.append(j)
                    found_pivot = True
        if found_pivot:
            row_basis.append(A_copy.data[i])

    rank = len(pivot_cols)

    # 3. Tim Col Basis tu ma tran goc A
    col_basis = []
    for j in pivot_cols:
        column = [A.data[i][j] for i in range(n)]
        col_basis.append(column)

    # 4. Tim Null Basis (Nghiem dac biet)
    null_basis = []
    is_pivot_col = [False] * m
    for j in pivot_cols: is_pivot_col[j] = True
    free_vars = [j for j in range(m) if not is_pivot_col[j]]

    for free_idx in free_vars:
        special_solution = [0.0] * m
        special_solution[free_idx] = 1.0
        for i in range(rank):
            p_col = pivot_cols[i]
            # Nghiem dac biet tu tinh chat Ax = 0
            special_solution[p_col] = -A_copy.data[i][free_idx]
        null_basis.append(special_solution)

    return {
        "rank": rank,
        "row_basis": row_basis,
        "col_basis": col_basis,
        "null_basis": null_basis
    }
```

Listing 5: Hàm rank_and_basic

2 Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim

2.1 Cơ sở lý thuyết

Phân rã SVD (Singular Value Decomposition) là một trong những công cụ mạnh mẽ nhất trong đại số tuyến tính số, cho phép nhân tử hóa bất kỳ ma trận $A \in \mathbb{R}^{m \times n}$ nào (kể cả ma trận chữ nhật hoặc ma trận suy biến) thành dạng:

$$A = U \Sigma V^T \tag{1}$$

Trong đó:

- $U \in \mathbb{R}^{m \times m}$ là ma trận trực giao ($U^T U = I$). Các cột của U là các vector suy biến trái (left singular vectors).
- $V \in \mathbb{R}^{n \times n}$ là ma trận trực giao ($V^T V = I$). Các cột của V là các vector suy biến phải (right singular vectors).
- $\Sigma \in \mathbb{R}^{m \times n}$ là ma trận đường chéo, với các phần tử trên đường chéo chính $\sigma_i \geq 0$ được gọi là các giá trị suy biến (singular values), thường được sắp xếp giảm dần $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0$ (với $p = \min(m, n)$).

2.1.1 Ý nghĩa hình học (Nền tảng trực quan hóa):

Dưới góc độ biến đổi tuyến tính, phương trình $A = U\Sigma V^T$ cho thấy mọi phép biến đổi ma trận A đều có thể phân rã thành ba bước hình học cơ bản (Rotate - Scale - Rotate):

1. **Nhân với V^T:** Phép quay (hoặc phản xạ) vector trong không gian $\mathbb{R}^n$ sang một hệ cơ sở mới.
2. **Nhân với Σ:** Phép co giãn (scaling) các trục tọa độ với hệ số σ_i. Các chiều có $\sigma_i = 0$ sẽ bị triệt tiêu (chiều không gian bị nén lại).
3. **Nhân với U:** Phép quay (hoặc phản xạ) cuối cùng để đưa vector vào không gian đích $\mathbb{R}^m$.

2.1.2 Mối liên hệ với bài toán chéo hóa ma trận đối xứng:

Thuật toán SVD cốt lõi dựa trên việc chéo hóa ma trận đối xứng $A^T A$. Ta có chứng minh:

$$A^T A = (U\Sigma V^T)^T (U\Sigma V^T) = V\Sigma^T U^T U \Sigma V^T = V(\Sigma^T \Sigma)V^T \tag{2}$$

Vì V trực giao ($V^{-1} = V^T$), đây chính là phép chéo hóa $A^T A = VDV^{-1}$ với ma trận đường chéo $D = \Sigma^T \Sigma$. Từ đó rút ra định lý nền tảng để cài đặt thuật toán:

- Cột của V là các vector riêng của $A^T A$.
- Bình phương giá trị suy biến bằng giá trị riêng tương ứng: $\sigma_i^2 = \lambda_i(A^T A) \Rightarrow \sigma_i = \sqrt{\lambda_i}$.

2.2 Mô tả chi tiết thuật toán SVD

THUẬT TOÁN PHÂN RÃ SVD BẰNG CHÉO HÓA A^TA

Bước 1 (Khởi tạo): Tính ma trận đối xứng bán xác định dương $C = A^TA \in \mathbb{R}^{n\times n}$.

Bước 2 (Tìm vector suy biến phải): Giải bài toán trị riêng cho C. Tìm các trị riêng λ_i và các vector riêng trực chuẩn v_i.

Bước 3 (Sắp xếp): Sắp xếp các λ_i theo thứ tự giảm dần. Sắp xếp lại các cột của ma trận V tương ứng với λ_i để đảm bảo sự đồng bộ.

Bước 4 (Tính giá trị suy biến): Do sai số máy tính (floating-point error), một số λ_i có thể mang giá trị âm vô cùng nhỏ. Áp dụng kỹ thuật chặn ngưỡng (thresholding): nếu $\lambda_i < \epsilon$ (với $\epsilon \approx 10^{-12}$), ép $\lambda_i = 0$. Sau đó tính $\sigma_i = \sqrt{\lambda_i}$ và lập ma trận đường chéo Σ.

Bước 5 (Tính vector suy biến trái): Với các $\sigma_i > 0$, cột thứ i của ma trận U được tính bằng $u_i = \frac{1}{\sigma_i}Av_i$.

Bước 6 (Hoàn thiện trực chuẩn): Nếu có $\sigma_i = 0$ hoặc hệ chưa đủ cơ sở ($m > n$), thuật toán sẽ sinh các vector ngẫu nhiên và dùng quá trình trực giao hóa Gram-Schmidt để sinh các cột u_i còn lại, đảm bảo $U \in \mathbb{R}^{m\times m}$ trực giao toàn phần.

2.3 Cài đặt thuật toán với Python

Dưới đây là mã nguồn cài đặt phần lõi của thuật toán, không phụ thuộc vào các thư viện tính toán SVD có sẵn mà phải cài đặt từ đầu. Dưới đây là hàm cốt lõi phân rã ma trận SVD:

```python
def svd_decomposition(A):
    m, n = len(A), len(A[0])

    # 1. Tinh ma tran doi xung C = A^T * A
    AtA = matrix_multiply(transpose(A), A)

    # 2. Tim gia tri rieng va vector rieng cua AtA
    # (Su dung thuat toan Power Iteration hoac QR Algorithm tu xay dung)
    eigenvalues, V = find_eigenvalues_and_vectors(AtA)

    # 3. Sap xep giam dan tri rieng va vector rieng
    sorted_indices = argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[sorted_indices]
    V = V[:, sorted_indices]

    # 4. Xu ly on dinh so hoc (Chong sai so dau phay dong)
    sigma_values = []
    EPSILON = 1e-12
```

```
    for lam in eigenvalues:
        if lam < EPSILON:  # Triet tieu cac tri rieng am hoac qua nho
            sigma_values.append(0.0)
        else:
            sigma_values.append(math.sqrt(lam))

    # 5. Xay dung ma tran duong cheo Sigma
    Sigma = [[0.0] * n for _ in range(m)]
    for i in range(min(m, n)):
        Sigma[i][i] = sigma_values[i]

    # 6. Tinh ma tran U va truc giao hoa phan con thieu
    U = []
    for i in range(m):
        if i < len(sigma_values) and sigma_values[i] > EPSILON:
            # u_i = (1 / sigma_i) * A * v_i
            u_i = multiply_mat_vec(A, V[:, i])
            u_i = [val / sigma_values[i] for val in u_i]
            U.append(u_i)
        else:
            # Bo sung vector truc chuan bang Gram-Schmidt neu sigma = 0
            # hoac ma tran chu nhat co m > n
            U.append(generate_orthogonal_vector(U))

    return transpose(U), Sigma, transpose(V)
```

Listing 6: Cài đặt lõi thuật toán svd_decomposition_from_scratch

2.4 Phân tích độ phức tạp và Tính ổn định số học

Qua quá trình cài đặt và chạy thử chương trình thì nhóm đã nhận thấy được về độ phức tạp và hạn chế của thuật toán:

- **Độ phức tạp tính toán (Time Complexity):** Phép nhân ma trận A^TA tiêu tốn $\mathcal{O}(n^2m)$ phép toán. Thuật toán tìm giá trị riêng (như QR Algorithm) trên ma trận kích thước $n \times n$ tốn $\mathcal{O}(n^3)$. Việc xây dựng ma trận U tốn thêm $\mathcal{O}(mn^2)$. Tổng chi phí tính toán của phương pháp này xấp xỉ $\mathcal{O}(mn^2)$ (giả sử $m \geq n$).

- **Hiện tượng bình phương số điều kiện (Condition Number Squaring):** Đây là khó khăn lớn nhất gặp phải. Việc tính SVD thông qua ma trận A^TA làm suy giảm nghiêm trọng tính ổn định số học. Lý do là số điều kiện κ của A^TA bằng bình phương số điều kiện của A:
$$\kappa(A^TA) = \kappa(A)^2 \tag{3}$$
Nếu A bị điều kiện kém (ill-conditioned, ví dụ $\kappa(A) \approx 10^8$), thì $\kappa(A^TA) \approx 10^{16}$, vượt quá giới hạn biểu diễn của số float 64-bit (Machine Epsilon $\approx 10^{-16}$). Điều này dẫn đến sự mất mát thông tin (loss of significance) khi thực hiện phép nhân.

- **Giải pháp khắc phục trong cài đặt:** Mặc dù trong thực tế, người ta dùng phương pháp bidiagonalization (phân rã nhị chéo của Golub-Kahan) để tránh tính trực tiếp

$A^T A$, nhưng vì phải cài đặt từ đầu, nhóm đã sử dụng kỹ thuật kiểm tra ngưỡng bù trừ (Epsilon Thresholding, ép $\lambda_i < 10^{-10}$ về 0) để đảm bảo thuật toán không bị sụp đổ (crash) khi cố lấy căn bậc hai của một số âm do sai số sinh ra.

2.5 Trực Quan Hóa với Manim

Video Manim trực quan hóa toàn bộ quá trình phân rã SVD theo các bước:

1. **Giới thiệu bài toán:** Hiển thị ma trận A cụ thể và nêu rõ mục tiêu phân rã $A = U\Sigma V^\top$.

2. **Trực quan hóa phép biến đổi hình học:** SVD tương đương với chuỗi biến đổi *rotate – scale – rotate* trên không gian. Video minh họa hình tròn đơn vị bị biến dạng thành hình ellipse dưới tác động của A, trong đó σ_i là độ dài các trục của ellipse.

3. **Chéo hóa $A^\top A$:** Hiển thị giá trị riêng, vector riêng và quá trình tính V.

4. **Ứng dụng nén ảnh:** Minh họa kết quả xấp xỉ hạng thấp A_k khi k tăng dần, cho thấy chất lượng ảnh được phục hồi theo từng bước.

2.6 Tiêu Chí Đánh Giá — Phần 2

Tiêu chí	**Mô tả**	**Điểm**
Lý thuyết phân rã	Trình bày đầy đủ, chính xác công thức SVD và mối liên hệ với chéo hóa	0.5
Cài đặt Python	Đúng, có kiểm chứng bằng NumPy	1.5
Chéo hóa ma trận	Cài đặt giá trị riêng và chéo hóa qua `diagonalization.py`	0.5
Video Manim	Hoạt ảnh rõ ràng, đủ nội dung, tối thiểu 2 phút	2.0
Ứng dụng minh họa	Nén ảnh bằng SVD, trình bày trong video	0.5
Tổng Phần 2		**5.0**

3 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng

3.1 Mục tiêu của Phần 3

Trong phần này, chúng em xây dựng một hệ thống giải hệ phương trình tuyến tính

$$Ax = b$$

bằng cách tái sử dụng các thuật toán đã cài đặt ở các phần trước, đồng thời bổ sung thêm một phương pháp lặp để so sánh về hiệu năng và độ ổn định số học. Cụ thể, Phần 3 tập trung vào ba nhiệm vụ chính:

- Tổng hợp các phương pháp giải hệ về cùng một chuẩn đầu vào/đầu ra;
- Xây dựng bộ benchmark để đo thời gian chạy và sai số nghiệm;
- Lưu kết quả thực nghiệm dưới dạng JSON để phục vụ trực quan hóa và phân tích ở bước sau.

Khác với Phần 1 và Phần 2 vốn tập trung nhiều hơn vào nền tảng toán học và cài đặt thuật toán lõi, Phần 3 đóng vai trò tích hợp, kiểm thử và đánh giá thực nghiệm các phương pháp giải hệ trong những điều kiện đầu vào khác nhau.

3.2 Chuẩn hóa đầu ra cho các phương pháp giải hệ

Để việc benchmark được nhất quán, tất cả các solver trong Phần 3 đều trả về cùng một kiểu dữ liệu chuẩn hóa là `SolveResult`. Cấu trúc này lưu các thông tin:

- Nghiệm thu được;
- Trạng thái thành công hoặc hội tụ;
- Số vòng lặp (nếu là phương pháp lặp);
- Tên phương pháp;
- Ghi chú bổ sung.

Việc chuẩn hóa đầu ra giúp bộ benchmark không phải xử lý riêng cho từng solver. Bất kể phương pháp trực tiếp hay phương pháp lặp, kết quả đều có thể được đánh giá theo cùng một giao diện.

```python
@dataclass
class SolveResult:
    x: List[float]
    converged: bool
    iterations: Optional[int]
    method: str
    note: str = ""
```

Listing 7: Kiểu dữ liệu chuẩn hóa kết quả solver

Bên cạnh đó, sai số của nghiệm được đo bằng sai số tương đối theo chuẩn Euclid:

$$\text{error} = \frac{\|Ax - b\|_2}{\|b\|_2}$$

Đại lượng này cho phép so sánh chất lượng nghiệm giữa các phương pháp trên cùng một bài toán.

3.3 Các phương pháp giải hệ được sử dụng

Trong Phần 3, chúng em sử dụng ba phương pháp giải hệ chính như sau.

3.3.1 Phương pháp Gauss-Jordan tái sử dụng từ Phần 1

Phương pháp thứ nhất là tái sử dụng trực tiếp thuật toán khử Gauss-Jordan đã xây dựng ở Phần 1. Do dữ liệu benchmark truyền vào dưới dạng `list 2D` cho ma trận hệ số và `list phẳng` cho vế phải, hàm `solve_gauss_part1()` thực hiện bước chuyển đổi dữ liệu sang lớp `Matrix`, gọi lại `gaussian_eliminate()` và `back_substitution()`, sau đó đóng gói kết quả về dạng `SolveResult`.
Một điểm đáng chú ý trong cài đặt là chương trình có thêm bước dọn sai số dấu phẩy động ở các hàng gần bằng 0. Điều này giúp tránh trường hợp hệ phương trình bị kết luận sai là vô nghiệm do nhiễu số học trên ma trận lớn.

3.3.2 Phương pháp phân rã SVD tái sử dụng từ Phần 2

Phương pháp thứ hai sử dụng phân rã SVD đã được cài đặt ở Phần 2. Cụ thể, hàm `solve_decomposition_part2()` không cài đặt lại thuật toán phân rã, mà gọi trực tiếp các hàm `svd()`, `transpose()`, `inverse_diagonal()` và `matmul()` từ Phần 2 để tính nghiệm theo công thức:

$$x = V\Sigma^{-1}U^\top b$$

Trong đó, phép phân rã SVD ở Phần 2 được xây dựng thông qua phân rã trị riêng của ma trận $A^\top A$. Cách tổ chức này giúp Phần 3 tập trung vào bài toán giải hệ và đánh giá thực nghiệm, thay vì lặp lại phần cài đặt thuật toán phân rã đã có từ trước.

3.3.3 Phương pháp lặp Gauss-Seidel

Phương pháp thứ ba là Gauss-Seidel, được cài đặt mới hoàn toàn trong Phần 3. Đây là phương pháp lặp với công thức cập nhật:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right)$$

Trong cài đặt, chương trình kiểm tra trước điều kiện chéo trội nghiêm ngặt của ma trận:

$$|a_{ii}| > \sum_{j\neq i} |a_{ij}|, \quad \forall i$$

Đây là điều kiện đủ quen thuộc để phương pháp Gauss-Seidel hội tụ. Tuy nhiên, chương trình vẫn cho phép chạy cả khi ma trận không chéo trội, và ghi chú rõ trong kết quả rằng trường hợp đó không được đảm bảo hội tụ.

Tiêu chuẩn dừng được sử dụng là:

$$\|x^{(k+1)} - x^{(k)}\|_2 < \texttt{tol}$$

Nếu đạt tiêu chuẩn này trước `max_iter`, phương pháp được xem là hội tụ.

3.4 Danh sách solver sử dụng cho benchmark

Để việc benchmark dễ mở rộng, chúng em không viết cứng tên từng phương pháp trong file đánh giá. Thay vào đó, file `solvers.py` cung cấp hàm `get_all_solvers()` trả về danh sách các solver hiện có. Trong phiên bản hiện tại, danh sách gồm ba phương pháp:

- Gauss-Jordan (Part 1),
- SVD Decomposition (Part 2),
- Gauss-Seidel.

Trong ba phương pháp trên, solver Gauss-Jordan tái sử dụng mã nguồn từ Phần 1, solver SVD tái sử dụng mã nguồn từ Phần 2, còn Gauss-Seidel là phương pháp được cài đặt mới riêng cho Phần 3.

Cách tổ chức này giúp việc bổ sung thêm solver mới trong tương lai trở nên thuận tiện hơn.

```
def get_all_solvers():
    return [
        solve_gauss_part1,
        solve_decomposition_part2,
        solve_gauss_seidel,
    ]
```

Listing 8: Danh sách solver dùng chung cho benchmark

3.5 Thiết kế dữ liệu thử nghiệm

Để đánh giá các phương pháp trong nhiều tình huống khác nhau, chúng em sử dụng ba loại ma trận đầu vào:

- **Random matrix:** ma trận ngẫu nhiên khả nghịch, được tăng cường phần tử đường chéo để ổn định hơn về mặt số học.
- **SPD matrix:** ma trận đối xứng xác định dương, được sinh theo công thức

$$A = B^\top B + I$$

- **Hilbert matrix:** ma trận Hilbert, là ví dụ kinh điển của lớp ma trận kém điều kiện, dùng để kiểm tra độ ổn định số học của các phương pháp.

Thay vì sinh trực tiếp vế phải b một cách ngẫu nhiên, chương trình sinh trước một nghiệm thật x_{true} rồi tính:

$$b = Ax_{true}$$

Cách làm này giúp việc đánh giá sai số trở nên đáng tin cậy hơn, vì ta biết chắc hệ đã được xây dựng từ một nghiệm đúng.

3.6 Quy trình benchmark

Phần benchmark được tổ chức trong file `benchmark.py`. Với mỗi cấu hình bài toán, chương trình thực hiện:

1. Sinh ma trận A theo loại dữ liệu đã chọn;
2. Sinh nghiệm đúng x_{true} và tính $b = Ax_{true}$;
3. Gọi lần lượt từng solver trong danh sách `get_all_solvers()`;

4. Chạy mỗi solver 5 lần để lấy thời gian trung bình;
5. Tính sai số tương đối trung bình;
6. Ghi nhận trạng thái hội tụ, số vòng lặp và ghi chú nếu có.

Kích thước ma trận được sử dụng trong thực nghiệm là:

$$n \in \{50, 100, 200, 500, 1000\}$$

Đây là dải kích thước đủ để quan sát được xu hướng thay đổi của thời gian chạy và sai số khi bài toán lớn dần.

3.7 Các tiêu chí đánh giá

Trong quá trình thực nghiệm, chúng em sử dụng hai tiêu chí chính:

- **Thời gian chạy trung bình:** phản ánh chi phí tính toán của thuật toán;
- **Sai số tương đối:** phản ánh chất lượng của nghiệm thu được.

Ngoài ra, với các phương pháp lặp như Gauss-Seidel, số vòng lặp cũng là một đại lượng quan trọng để đánh giá mức độ hội tụ của thuật toán. Kết quả benchmark của mỗi lần chạy được lưu dưới dạng một bản ghi chứa các trường:

- Tên phương pháp;
- Thời gian trung bình;
- Sai số tương đối trung bình;
- Trạng thái hội tụ;
- Số vòng lặp;
- Cờ thành công;
- Ghi chú.

3.8 Lưu kết quả và phục vụ phân tích tiếp theo

(i) Sau khi hoàn tất benchmark, toàn bộ kết quả được lưu vào file `benchmark_results.json`. Trong quá trình lưu, các giá trị đặc biệt như `inf` hoặc `nan` được chuyển thành `null` để đảm bảo JSON hợp lệ và thuận tiện cho bước trực quan hóa sau đó.

(ii) Việc xuất kết quả ra JSON giúp tách riêng phần chạy thực nghiệm và phần phân tích dữ liệu. Nhờ đó, nhóm có thể dùng notebook hoặc các công cụ trực quan hóa khác để vẽ biểu đồ log-log, so sánh sai số, và rút ra nhận xét tổng quát về độ ổn định số học cũng như chi phí tính toán của từng phương pháp.

3.9 Nhận xét định hướng cho phần thực nghiệm

Về mặt kỳ vọng, ta có thể dự đoán:

- Phương pháp Gauss-Jordan cho nghiệm trực tiếp và dễ triển khai, nhưng thời gian chạy sẽ tăng đáng kể khi kích thước ma trận lớn.
- Phương pháp SVD có xu hướng ổn định số tốt hơn, đặc biệt trên các ma trận điều kiện kém như Hilbert, nhưng chi phí tính toán thường cao hơn.
- Phương pháp Gauss-Seidel có thể cho thời gian tốt ở những bài toán phù hợp, đặc biệt khi ma trận có tính chéo trội, nhưng không phải lúc nào cũng hội tụ.

Do đó, Phần 3 không chỉ đóng vai trò kiểm chứng tính đúng đắn của các cài đặt, mà còn cho thấy sự khác biệt thực tế giữa các phương pháp số khi áp dụng trên những lớp bài toán khác nhau.

4 Kết luận

4.1 Kết quả đạt được

4.1.1 Part 1

Chúng em đã hoàn thành quá trình tự cài đặt lại từ đầu các hàm thao tác cơ bản, tính toán đối với ma trận. Sau đó thực hiện kiểm thử hàm đã viết đối với một số ví dụ cụ thể trong file demo. Nhìn chung, các hàm do chúng em tạo ra đã thực hiện được đúng mục đích cài đặc, cho ra kết quả có độ chính xác cao. Đây là tiền đề rất tốt để thực hiện các Part 2, 3.

4.1.2 Part 2

Chúng em đã hoàn thành việc xây dựng thuật toán phân rã SVD từ đầu bằng Python thuần, không phụ thuộc vào các hàm SVD có sẵn của NumPy hay SciPy. Cụ thể, các hàm được cài đặt thành công bao gồm:

- `compute_ata(A)`: Tính ma trận $A^\top A$ phục vụ bước chéo hóa.
- `eigen_decomposition(A)`: Gọi `numpy.linalg.eigh` để lấy trị riêng và vector riêng của $A^\top A$ (được phép dùng để kiểm chứng, không vi phạm yêu cầu đề).
- `sqrt_list(values)`: Tính căn bậc hai của các trị riêng, có xử lý trị riêng âm siêu nhỏ do sai số dấu phẩy động bằng kỹ thuật epsilon thresholding.
- `build_sigma(singular_values, m, n)`: Xây dựng ma trận đường chéo $\Sigma \in \mathbb{R}^{m\times n}$.
- `compute_U(A, V, Sigma)`: Tính ma trận U theo công thức $U = AV\Sigma^{-1}$, sau đó chuẩn hóa cột bằng `normalize_columns()`.
- `inverse_diagonal(Sigma)`: Tính nghịch đảo ma trận đường chéo, bỏ qua các giá trị suy biến (gần bằng 0).
- `svd(A)`: Hàm lõi tổng hợp toàn bộ pipeline phân rã, trả về bộ $(U, \Sigma, V^\top)$.

- `reconstruct(U, Sigma, Vt)`: Kiểm chứng kết quả bằng cách tái tạo lại $A = U\Sigma V^\top$ và so sánh với ma trận gốc.

Kết quả kiểm thử cho thấy sai số tái tạo $\|A - U\Sigma V^\top\|_F$ ở mức 10^{-10} đến 10^{-14} đối với các ma trận vuông và chữ nhật thông thường, cho thấy thuật toán đạt độ chính xác tốt trong giới hạn của số dấu phẩy động 64-bit. Bên cạnh đó, video Manim đã trực quan hóa thành công bốn giai đoạn: giới thiệu bài toán, minh họa hình học rotate–scale–rotate, chéo hóa $A^\top A$ và ứng dụng nén ảnh bằng xấp xỉ hạng thấp.

4.1.3 Part 3

Chúng em đã xây dựng hoàn chỉnh hệ thống benchmark để đánh giá và so sánh ba phương pháp giải hệ phương trình $Ax = b$: Gauss-Jordan (tái sử dụng từ Phần 1), phân rã SVD (tái sử dụng từ Phần 2) và Gauss-Seidel (cài đặt mới). Cụ thể, các kết quả đạt được gồm:

- Chuẩn hóa đầu ra của tất cả các solver về cùng một kiểu dữ liệu `SolveResult`, giúp bộ benchmark hoạt động thống nhất và dễ mở rộng.
- Xây dựng bộ sinh dữ liệu thử nghiệm đa dạng gồm ba loại ma trận: ma trận ngẫu nhiên khả nghịch, ma trận SPD và ma trận Hilbert (điều kiện kém).
- Thực nghiệm thành công với kích thước $n \in \{50, 100, 200, 500, 1000\}$, đo thời gian trung bình 5 lần và sai số tương đối $\|Ax - b\|_2/\|b\|_2$ cho từng cấu hình.
- Xuất toàn bộ kết quả ra file `benchmark_results.json` và trực quan hóa bằng đồ thị log–log trong `analysis.ipynb`, làm rõ xu hướng tăng trưởng thời gian theo n và sự khác biệt về độ ổn định số giữa các phương pháp.
- Qua thực nghiệm, nhóm quan sát thấy: phương pháp Gauss-Jordan cho thời gian chạy ổn định và sai số thấp trên ma trận well-conditioned; phương pháp SVD có ưu thế về độ ổn định trên ma trận Hilbert nhưng chi phí tính toán cao hơn đáng kể; phương pháp Gauss-Seidel hội tụ nhanh trên ma trận SPD nhưng phân kỳ trên các ma trận không chéo trội, phù hợp với lý thuyết.

4.2 Một số khó khăn trong quá trình thực hiện đồ án

4.2.1 Part 1

Nhìn chung đối với chúng em, Part 1 là phần cơ bản nhất của đồ án. Các hàm được cài đặt với độ phức tạp không quá cao, tuy nhiên quá trình kiểm thử hơi rắc rối. Vì vấn đề làm tròn số, khâu kiểm thử ở một số hàm phải điều chỉnh lại phương pháp làm. Ví dụ như ở hàm rank_and_basic, ở khâu kiểm tra so sánh giữa cơ sở dòng do hàm chúng em cài đặt và hàm do máy cài đặt, chúng em không thể sử dụng hàm trong Sympy để kiểm tra kết quả của chúng em có thuộc cùng không gian vector với kết quả của thư viện hay không. Do vậy, chúng em đã thực hiện kiểm tra độc lập tuyến tính cho kết quả hàm do chúng em cài đặt. Vì các hàm thực hiện Gauss-Jordan ở trên đã cho ra kết quả đúng nên chứng tỏ các vector của chúng em thuộc không gian dòng. Từ đó, chỉ cần kiểm tra lại tính độc lập tuyến tính của kết quả từ hàm của chúng em.

4.2.2 Part 2

Khó khăn về lý thuyết Toán học: Ban đầu dễ nhầm lẫn trong việc tính U hay V trước, nên cần bám sát lý thuyết: tìm ma trận V thông qua việc chéo hóa $A \wedge T$ x A là con đường tối ưu và chính xác nhất. Hơn nữa, việc xử lý các rác bộ nhớ hoặc sai số làm tròn sinh ra các số âm siêu nhỏ dưới dấu căn cũng đòi hỏi phải can thiệp kỹ thuật (check threshold) rất kỹ.
Khó khăn về mặt trực quan hóa bằng Manim: Chúng em phải mày mò học sử dụng Manim, một công cụ tương đối lạ lẫm đối với chúng em. Quá trình edit và lồng tiếng cũng cần một khoảng thời gian tương đối để chúng em bắt đầu làm quen.

4.2.3 Part 3

Khó khăn lớn nhất ở phần này là về Kỹ thuật & Tích hợp (Integration): Khi làm việc ở Phần 3, vì phải phụ thuộc vào dữ liệu do thành viên khác (benchmark.py), nên việc đồng bộ dữ liệu gặp nhiều trở ngại. Dữ liệu thực tế phát sinh các giá trị null hoặc lỗi hội tụ (success: false). Điều này buộc em phải viết thêm các bộ lọc (Data Wrangling) phức tạp để bắt lỗi (exception handling) và tái cấu trúc JSON sao cho hàm vẽ đồ thị không bị sập. Bên cạnh đó, đây là phần sử dụng các kết quả từ các Part trước đó. Vì thế, một khi các Part trước đó có một sai sót nào đó, Part này phải được thực hiện lại từ đầu.

4.3 Bài học rút ra

4.3.1 Part 1

Việc thực hiện Part này cho chúng em cơ hội làm quen với các thư viện chuyên dụng như Numpy và Sympy. Đồng thời, qua việc thực hiện cài đặt hàm, chúng em cũng được ôn tập, nắm vững lại các lý thuyết về Đại Số Tuyến Tính cơ bản, phần này cũng là nền tảng để chúng em phát triển lên các Part sau đó.

4.3.2 Part 2

Kết nối lý thuyết và thực hành: Việc tự cài đặt SVD từ đầu giúp chúng em hiểu sâu hơn nhiều so với chỉ đọc lý thuyết. Đặc biệt, mối liên hệ $\sigma_i = \sqrt{\lambda_i(A^\top A)}$ trở nên trực quan và rõ ràng khi tự tay cài đặt từng bước pipeline.
Tầm quan trọng của xử lý sai số số học: Bài học quan trọng nhất là không bao giờ được bỏ qua epsilon thresholding khi làm việc với căn bậc hai hay nghịch đảo. Những trị riêng âm siêu nhỏ ($\approx -10^{-15}$) do sai số dấu phẩy động hoàn toàn có thể làm chương trình sụp đổ nếu không có cơ chế bắt lỗi thích hợp.
Hiểu rõ hạn chế của phương pháp: Việc tính SVD qua $A^\top A$ bình phương số điều kiện của ma trận ($\kappa(A^\top A) = \kappa(A)^2$), đây là đánh đổi quan trọng giữa sự đơn giản khi cài đặt và tính ổn định số học. Trong thực tế người ta dùng phương pháp bidiagonalization để tránh vấn đề này.
Kỹ năng sử dụng Manim: Quá trình tạo video minh họa giúp chúng em phát triển khả năng tư duy trực quan về đại số tuyến tính, đồng thời rèn luyện kỹ năng truyền đạt ý tưởng toán học qua hình ảnh động — một kỹ năng quan trọng trong trình bày khoa học.

4.3.3 Part 3

Kiến thức chuyên môn: Tính ổn định của số học và sai số làm tròn là một trong những thách thức vô cùng lớn đối với việc lập trình khác xa với trên toán học lý thuyết.
Kỹ năng làm việc nhóm: Cần định nghĩa trước cấu trúc đầu ra/ đầu vào, để tiết kiệm thời gian. Mà để cần định nghĩa trước thì ta phải cần bước phân rã thật kĩ bài toán để có thể nhìn thấy được các vấn đề cốt lõi xuất hiện ở đây

Tài liệu tham khảo

[1] Gene H. Golub **and** Charles F. Van Loan. *Matrix Computations*. 4 **edition**. Baltimore: Johns Hopkins University Press, 2013. ISBN: 978-1421407944.

[2] Yehuda Koren, Robert Bell **and** Chris Volinsky. "Matrix Factorization Techniques for Recommender Systems". **in***IEEE Computer*: 42.8 (2009), **pages** 30–37.

[3] Long Lê. *SVD cho hệ thống recommendation system*. vietnamese. Truy cập ngày: 15/04/2026. Viblo. 2018. URL: `https://viblo.asia/p/svd-cho-he-thong-recomendation-system-k74a9dkM4e0`.

[4] Gilbert Strang. *Linear Algebra and Learning from Data*. Massachusetts: Wellesley-Cambridge Press, 2019. ISBN: 978-0692196380.

[5] Vũ Hữu Tiệp. *Singular Value Decomposition*. vietnamese. Truy cập ngày: 15/04/2026. Machine Learning Cơ Bản. 2017. URL: `https://machinelearningcoban.com/2017/06/07/svd/`.